

Assignment - Senior Backend Developer

Envobyte Ltd. — Technical Assignment

Tech Stack: PHP / Laravel · MySQL 8.0+ · Redis · Queue Workers

Time Estimate: 7–9 hours

Marks: 100

Existing Project Work: Monica CRM

Background

You will work with **Monica** — an open-source personal CRM built with Laravel. Monica helps people organize their social relationships by tracking contacts, activities, reminders, notes, journal entries, and calls.

Project URL: <https://github.com/monicahq/monica>

Monica has grown from a small side project to serving thousands of self-hosted instances and a hosted SaaS platform with millions of contacts. As a result, several architectural decisions that made sense early on are now creating bottlenecks. Your task is to analyze the codebase deeply, make architectural trade-offs, and ship a significant change that improves the system's reliability, observability, or scalability.

Setup

1. Fork the Monica repository
 2. Set it up locally following the project's documentation
 3. Create a new branch named `envobyte-assignment`
-

Project: Reliable Background Import System

Monica allows users to **import contacts** from a CSV or vCard file. The current implementation processes imports synchronously during the HTTP request. For files with more than 100 contacts, this causes request timeouts and a poor user experience. The import also has no progress tracking — the user sees a loading spinner and has no idea how many contacts have been processed or if the import failed.

Your project is to redesign the import system end-to-end: from file upload to background processing, with progress tracking, error handling, and observability.

Problem Analysis

Explore the existing import implementation before designing your solution. Key files to investigate:

- `app/Http/Controllers/ImportController.php` or similar
- Any import job or service classes
- The `import_jobs` or equivalent table schema
- The CSV/vCard parsing logic

Document the current flow: what happens from the moment a user uploads a file to when contacts are created.

Requirements

1. Background Processing Pipeline

- Move the import processing to a **background job** (Laravel queue)
- The HTTP endpoint should accept the file, validate it, create an `import_jobs` record with status `pending`, and dispatch the job — all within <500ms
- The job should process contacts in **batches** (e.g., 50 contacts per batch) to avoid memory exhaustion on large files
- Each batch is a separate job that updates the `import_jobs` progress (e.g., "120/500 contacts processed")
- If any batch fails, the overall import status should become `failed` and remaining batches should not start

Design the `import_jobs` table schema:

Column	Type	Purpose
<code>id</code>		Primary key
<code>account_id</code>		For sharding/scoping
<code>user_id</code>		Who initiated the import
<code>filename</code>		Original uploaded filename
<code>total_rows</code>		Count after CSV parsing
<code>processed_rows</code>		Incremented by batch jobs

<code>failed_rows</code>	Count of rows that failed	
<code>status</code>	<code>pending</code> , <code>processing</code> , <code>completed</code> , <code>failed</code>	
<code>errors</code>	JSON	Array of per-row error messages
<code>started_at</code>	When processing began	
<code>completed_at</code>	When processing finished	

Add any additional columns you need. Justify each addition.

2. Progress Tracking API

Method	Path	Description
GET	<code>/api/import</code>	List recent imports for the user (paginated, with status and progress %)
GET	<code>/api/import/:id</code>	Get detailed status: progress %, processed count, failed count, estimated time remaining, errors so far
POST	<code>/api/import/:id/cancel</code>	Cancel a running import — remaining batches should not execute
GET	<code>/api/import/:id/errors</code>	Paginated list of per-row errors with row number and error message

The progress endpoint should be **fast** (<50ms) — it should not query the contact rows, only the `import_jobs` record.

Sample Input/Output

The following examples illustrate expected behaviour.

POST `/api/import` — Upload a CSV file for import

```
Request:  POST /api/import
Headers:  Content-Type: multipart/form-data
Body:     file=contacts.csv (CSV attachment)
```

```
Response: 201
{
```

```
{
  "data": {
    "id": 1,
    "filename": "contacts.csv",
    "total_rows": 500,
    "processed_rows": 0,
    "failed_rows": 0,
    "status": "pending",
    "created_at": "2026-06-03T12:00:00Z"
  }
}
```

GET /api/import/1 — Get import progress

Request: GET /api/import/1

Response: 200

```
{
  "data": {
    "id": 1,
    "filename": "contacts.csv",
    "total_rows": 500,
    "processed_rows": 320,
    "failed_rows": 2,
    "status": "processing",
    "progress_pct": 64,
    "errors": [
      { "row": 42, "message": "Invalid email: 'not-an-email'" },
      { "row": 87, "message": "Missing required field: name" }
    ],
    "started_at": "2026-06-03T12:00:05Z",
    "estimated_remaining_sec": 45
  }
}
```

GET /api/import/1/errors.csv — Download error CSV

Request: GET /api/import/1/errors.csv

Response: 200 (CSV content)

```
name,email,phone,error
"John Doe",not-an-email,,Invalid email: 'not-an-email'
,Jane,,Missing required field: name
```

GET /api/import — List recent imports

Request: GET /api/import?page=1&per_page=10

```
Response: 200
{
  "data": [
    {
      "id": 1,
      "filename": "contacts.csv",
      "total_rows": 500,
      "processed_rows": 500,
      "failed_rows": 2,
      "status": "completed",
      "progress_pct": 100,
      "created_at": "2026-06-03T12:00:00Z"
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 10,
    "total": 1,
    "last_page": 1
  }
}
```

3. Batch Processing with Error Isolation

Within the import job, implement per-row error handling:

- If a single row fails validation (e.g., missing email, invalid phone), skip that row but continue processing the rest
- Record the error with the row number and error message in `import_jobs.errors`
- After all batches complete, the overall status is `completed` even if some rows failed — unless ALL rows failed
- Provide a downloadable error CSV: `GET /api/import/:id/errors.csv` — returns a CSV with original row data plus an error column

Write the query or logic for generating the error CSV. How do you reconstruct the original row data?

4. Concurrency & Idempotency

- What happens if the user uploads the same file twice? Should the system detect duplicates? Propose a mechanism (e.g., hash the file content, compare filename + size + date, or just let both run).
- What happens if a batch job crashes mid-way? The batch was marked as `processing` but never completed. Design a recovery mechanism that detects and retries stuck batches.
- If the user cancels an import while a batch is actively running, how do you ensure no new batches start and the running batch is stopped gracefully?

5. Observability & Alerting

Propose monitoring for the import system:

- What metrics would you track? (e.g., imports started, imports completed, imports failed, avg processing time per row, batch failure rate)
 - Write a query that detects "stuck" imports — imports that have been in `processing` status for more than 30 minutes
 - Propose an alerting rule: if the import failure rate (failed rows / total rows) exceeds 20% across all imports in the last hour, notify the team
-

Tests Required

- Submission **must** include automated tests covering the main functionality:
 - Import initiation and status tracking
 - Batch processing with per-row error handling
 - Import cancellation
 - Error CSV generation
 - Cache/invalidation behaviour where applicable
- Tests must be runnable via a single command (e.g., `php artisan test`)
- README must contain instructions to run the tests
- Submissions without runnable tests may be considered incomplete

What We Are Evaluating

Area	Weight
Understanding of existing codebase & problem analysis	10%
Queue architecture & batch processing design	25%
API design for progress tracking	15%
Error handling & isolation	15%
Concurrency, idempotency & recovery	20%
Production awareness (monitoring, alerting, edge cases)	15%

Strong senior candidates will:

- Identify and document issues in the current implementation that the requirements didn't mention
 - Discuss trade-offs explicitly — e.g., batching vs streaming, database-polling vs event-driven progress, one-queue vs per-import queues
 - Show production awareness: what happens at 10x scale, how to roll back, how to debug a stuck import
 - Question the requirements where appropriate — e.g., "should we store the entire CSV in the database or on S3?"
 - Write code that is easy to remove later (feature flags, clear interfaces)
-

Submission Requirements

- Create a **public GitHub repository** containing your changes
 - The repository should include:
 - A `README.md` with your approach, assumptions, trade-offs, and test instructions
 - All migration files, code files, and tests
 - A short **architecture decision record (ADR)** in `README.md` for at least two significant decisions (e.g., "Use database for progress tracking vs Redis" or "Batch size selection and why")
 - Do **not** fork Monica into your repo — just your changes on top of the original
 - Submit your assignment using the following form: [Assignment Submit Form Link](#)
-